

Université Assane Seck de Ziguinchor

Faculté des Sciences – Département d'Informatique

Rapport Intermédiaire

Conception et Déploiement d'une Architecture SDN

pour la Gestion Dynamique du Trafic et la Sécurité Réseau

Module : Réseaux Avancés / Architectures Réseau

Niveau : Master 1 – Semestre 2

Année universitaire : 2025–2026

Étudiante : Aminata Thior

Encadrant : Pr. Malick Ndoeye

Date : 12 mai 2026

Table des matières

Introduction	2
1 État de l'Art	3
1.1 Limitations des réseaux traditionnels	3
1.2 Le paradigme SDN	3
1.3 Le protocole OpenFlow	4
1.3.1 Principe de fonctionnement	4
1.3.2 Versions du protocole	4
1.4 Comparaison des contrôleurs SDN open-source	5
1.5 Travaux existants sur la sécurité SDN	5
2 Architecture et Environnement de Simulation	7
2.1 Architecture proposée	7
2.2 Environnement logiciel	7
2.2.1 Installation et configuration	7
2.2.2 Configuration de Ryu sous Python 3.12	7
2.3 Topologie réseau de référence	8
2.3.1 Description de la topologie	8
2.3.2 Commande de déploiement	9
2.3.3 Validation de la connectivité	9
2.3.4 Vérification de la structure réseau	9
3 Planification de la Suite du Projet	11
3.1 Travaux réalisés	11
3.2 Planification des phases suivantes	11
Conclusion	12

Introduction

Ce rapport intermédiaire constitue le livrable L1 du projet de Master 1 portant sur la conception et le déploiement d'une architecture SDN (*Software-Defined Networking*) pour la gestion dynamique du trafic et la sécurité réseau.

Il couvre les deux premières phases du projet :

- **Phase 1** : Étude bibliographique et état de l'art sur le SDN, le protocole OpenFlow, et les contrôleurs SDN open-source.
- **Phase 2** : Mise en place de l'environnement de simulation et validation de la connectivité de base via OpenFlow 1.3.

L'objectif central de ce projet est de répondre à la question suivante : *Comment concevoir et implémenter une architecture SDN capable de gérer dynamiquement le trafic réseau tout en intégrant des mécanismes de détection et de mitigation automatique des attaques réseau ?*

Chapitre 1

État de l'Art

1.1 Limitations des réseaux traditionnels

Les réseaux IP traditionnels reposent sur une architecture distribuée dans laquelle chaque équipement réseau — routeur ou commutateur — intègre simultanément le **plan de contrôle** (prise de décision sur le routage) et le **plan de données** (transfert physique des paquets). Cette architecture monolithique présente plusieurs limitations fondamentales [2] :

- **Rigidité** : la configuration manuelle de chaque équipement rend le réseau difficile à adapter dynamiquement aux variations de charge ou aux nouvelles politiques.
- **Complexité de gestion** : l'administration de réseaux hétérogènes multi-constructeurs nécessite une expertise spécifique pour chaque type d'équipement.
- **Absence de vue globale** : chaque équipement ne dispose que d'une vision partielle du réseau, ce qui complique l'optimisation des performances et la détection d'anomalies.
- **Sécurité réactive** : les politiques de sécurité sont statiques et ne s'adaptent pas en temps réel aux menaces émergentes.

1.2 Le paradigme SDN

Le *Software-Defined Networking* (SDN) propose de résoudre ces limitations en **séparant le plan de contrôle du plan de données** [1]. Cette séparation introduit un contrôleur logiquement centralisé qui dispose d'une vision globale du réseau et programme dynamiquement les règles de transfert sur les commutateurs via une interface standardisée.

L'architecture SDN est organisée en trois couches distinctes :

Les interfaces entre ces couches sont :

Couche	Composants	Rôle
Application	Modules QoS, sécurité, tableau de bord	Logique métier réseau
Contrôle	Contrôleur SDN (Ryu, ONOS, ODL)	Gestion centralisée, décisions
Infrastructure	Commutateurs (OVS, Mininet)	OpenFlow Transfert des paquets

TABLE 1.1 – Architecture en couches du réseau SDN

- **Southbound API** : interface entre le contrôleur et les commutateurs (protocole OpenFlow).
- **Northbound API** : interface entre le contrôleur et les applications réseau (REST API, Python).

1.3 Le protocole OpenFlow

OpenFlow est le protocole standard de l'interface *southbound* du SDN. Introduit par McKeown *et al.* en 2008 [1], il permet au contrôleur de lire et modifier les **tables de flux** (*flow tables*) des commutateurs.

1.3.1 Principe de fonctionnement

Lorsqu'un paquet arrive sur un commutateur OpenFlow :

1. Le commutateur consulte sa *flow table* pour trouver une règle correspondante.
2. Si une règle existe, l'action associée est exécutée (transfert, abandon, modification).
3. Si aucune règle ne correspond, un message **Packet-In** est envoyé au contrôleur.
4. Le contrôleur calcule la règle appropriée et répond par un message **Flow-Mod**.
5. La règle est installée dans la *flow table* du commutateur.

1.3.2 Versions du protocole

OpenFlow a évolué de la version 1.0 (2009) à la version 1.5 (2014). Ce projet utilise **OpenFlow 1.3**, qui introduit notamment :

- Les compteurs de flux (**flow_stats**, **port_stats**) pour le monitoring.
- Les files d'attente pour la qualité de service (QoS).
- Le support des tables multiples (*pipeline* de traitement).

1.4 Comparaison des contrôleurs SDN open-source

Plusieurs contrôleurs SDN open-source sont disponibles. Nous les comparons selon les critères pertinents pour ce projet [3] :

Contrôleur	Langage	Points forts	Limitations	Usage
Ryu	Python	Simple, léger, bien documenté, idéal pour prototypage	Moins scalable	Ce projet
ONOS	Java	Très scalable, production-ready, haute disponibilité	Complexe à configurer	Grands réseaux
OpenDaylight	Java	Très complet, modulaire, large communauté	Lourd, courbe d'apprentissage élevée	Industrie
Floodlight	Java	API REST claire, stable	Communauté moins active	Usage académique

TABLE 1.2 – Comparaison des contrôleurs SDN open-source

Justification du choix de Ryu : Ryu est écrit en Python, langage maîtrisé dans ce projet, et offre une API claire pour le développement de modules personnalisés. Sa légèreté le rend particulièrement adapté à un environnement de simulation sur machine unique. Il est largement utilisé dans les travaux de recherche académique sur le SDN.

1.5 Travaux existants sur la sécurité SDN

La sécurité dans les réseaux SDN a fait l'objet de nombreux travaux. Nous relevons les plus pertinents pour ce projet :

- **Braga *et al.* (2010) [4]** proposent une méthode légère de détection d'attaques DDoS dans les réseaux OpenFlow, basée sur l'analyse de l'entropie des adresses IP source. Cette approche constitue la base de notre module de détection.
- **Shin *et al.* (2013) [5]** présentent AVANT-GUARD, un système de gestion des flux dans les commutateurs SDN qui résiste aux attaques par saturation du contrôleur (attaques SYN flood ciblant le canal de contrôle).
- **Kreutz *et al.* (2015) [2]** proposent une taxonomie complète des vecteurs d'attaque spécifiques aux réseaux SDN, incluant les attaques sur le plan de contrôle, le plan de données et les interfaces nord/sud.

Notre projet s'appuie sur ces travaux pour concevoir un module de détection d'anomalies basé sur l'analyse statistique du trafic, couplé à un mécanisme de mitigation automatique par insertion dynamique de règles de filtrage.

Chapitre 2

Architecture et Environnement de Simulation

2.1 Architecture proposée

L'architecture SDN retenue pour ce projet est organisée autour de trois composants principaux :

1. **Couche infrastructure** : topologie réseau émulée par Mininet, composée de commutateurs Open vSwitch (OVS) communiquant via OpenFlow 1.3.
2. **Couche contrôle** : contrôleur Ryu 4.34 hébergeant les modules applicatifs développés (gestion du trafic, sécurité).
3. **Couche application** : modules Python développés en s'appuyant sur l'API Ryu (gestion QoS, détection DDoS, pare-feu programmable).

2.2 Environnement logiciel

2.2.1 Installation et configuration

L'environnement de simulation a été installé sur Ubuntu 24.04 LTS (machine de l'Université Assane Seck de Ziguinchor). La liste complète des outils déployés est présentée dans le tableau suivant :

2.2.2 Configuration de Ryu sous Python 3.12

L'installation de Ryu 4.34 sous Python 3.12 a nécessité plusieurs ajustements en raison d'incompatibilités avec les versions récentes de `setuptools` et `eventlet`. Les correctifs appliqués sont documentés ci-dessous :

```
1 # Cr ation de l'environnement virtuel
```


Outil	Rôle	Version validée
Mininet	Émulation de la topologie réseau	2.3+
Ryu	Contrôleur SDN	4.34
Open vSwitch	Commutateur OpenFlow virtuel	OVS + OpenFlow13
iperf3	Génération et mesure du trafic	3.16
hping3	Simulation d'attaques réseau	3.0.0
Scapy	Génération de paquets personnalisés	2.7.0
Wireshark	Analyse du trafic réseau	4.2.2
Python	Développement des modules Ryu	3.12
Git	Gestion des versions du code	–

TABLE 2.1 – Outils et versions de l'environnement de simulation

```

2 python3 -m venv ~/projet-sdn/venv
3 source ~/projet-sdn/venv/bin/activate
4
5 # Downgrade de setuptools pour compatibilit avec ryu/hooks.py
6 pip install "setuptools==67.8.0"
7
8 # Installation de Ryu depuis les sources
9 git clone https://github.com/faucetsdn/ryu.git ryu-src
10 cd ryu-src && pip install .
11
12 # Correctifs des dépendances incompatibles avec Python 3.12
13 pip install --upgrade "eventlet>=0.35.0"
14 pip install "dnspython>=2.0.0"

```

Listing 2.1 – Installation de Ryu avec les correctifs nécessaires

2.3 Topologie réseau de référence

2.3.1 Description de la topologie

La topologie réseau déployée pour ce projet est une topologie en **arbre (tree topology)** avec une profondeur de 2 et un facteur de branchement de 4, ce qui correspond à une structure proche d'un fat-tree simplifié. Cette topologie comprend :

- **1 commutateur racine** (s1) : niveau cœur (*core*).
- **4 commutateurs feuilles** (s2, s3, s4, s5) : niveau accès.
- **16 hôtes** (h1 à h16) : 4 hôtes par commutateur d'accès.
- **1 contrôleur distant** (c0) : Ryu sur 127.0.0.1:6633.

2.3.2 Commande de déploiement

```

1 # Terminal 1 : lancer le contr leur Ryu
2 ryu-manager ryu.app.simple_switch_13
3
4 # Terminal 2 : d ployer la topologie
5 sudo mn --controller=remote,ip=127.0.0.1,port=6633 \
6         --switch=ovsk,protocols=OpenFlow13 \
7         --topo=tree,depth=2,fanout=4

```

Listing 2.2 – Déploiement de la topologie sous Mininet

2.3.3 Validation de la connectivité

La validation de la topologie a été effectuée via la commande `pingall` de Mininet, qui teste la connectivité entre toutes les paires d’hôtes. Les résultats obtenus sont présentés ci-dessous :

```

1 mininet> pingall
2 *** Ping: testing ping reachability
3 h1 -> h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
4 h2 -> h1 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15 h16
5 [...]
6 h16 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13 h14 h15
7 *** Results: 0% dropped (240/240 received)

```

Listing 2.3 – Résultat du test de connectivité (extrait)

Le résultat **0% de paquets perdus (240/240 reçus)** confirme que :

- OpenFlow 1.3 est correctement activé sur tous les commutateurs OVS.
- Le contrôleur Ryu installe correctement les règles de flux (*flow entries*) dans les tables des commutateurs.
- La communication entre le contrôleur et les commutateurs via le canal OpenFlow fonctionne correctement.

2.3.4 Vérification de la structure réseau

La structure de la topologie a été vérifiée via la commande `net` de Mininet :

```

1 mininet> net
2 s1 lo: s1-eth1:s2-eth5 s1-eth2:s3-eth5 s1-eth3:s4-eth5 s1-eth4:
   s5-eth5
3 s2 lo: s2-eth1:h1-eth0 s2-eth2:h2-eth0 s2-eth3:h3-eth0 s2-eth4:
   h4-eth0

```

```
4 s3 lo: s3-eth1:h5-eth0 s3-eth2:h6-eth0 s3-eth3:h7-eth0 s3-eth4:  
   h8-eth0  
5 s4 lo: s4-eth1:h9-eth0 s4-eth2:h10-eth0 s4-eth3:h11-eth0 s4-eth4  
   :h12-eth0  
6 s5 lo: s5-eth1:h13-eth0 s5-eth2:h14-eth0 s5-eth3:h15-eth0 s5-  
   eth4:h16-eth0  
7 c0 (RemoteController: 127.0.0.1:6633)
```

Listing 2.4 – Structure de la topologie (commande net)

Chapitre 3

Planification de la Suite du Projet

3.1 Travaux réalisés

Phase	Description	Statut
Phase 1	État de l'art SDN, OpenFlow, contrôleurs	Terminée
Phase 2	Installation environnement, validation topologie	Terminée

TABLE 3.1 – Avancement du projet

3.2 Planification des phases suivantes

Phase	Objectif	Durée
Phase 3	Module de gestion du trafic (QoS, load balancing, monitoring)	3 semaines
Phase 4	Module de sécurité (détection DDoS, mitigation, firewall)	3 semaines
Phase 5	Tests et évaluation des performances	3 semaines
Phase 6	Rédaction du rapport final et soutenance	2 semaines

TABLE 3.2 – Planification des phases restantes

Conclusion

Ce rapport intermédiaire présente les travaux réalisés lors des deux premières phases du projet SDN. L'état de l'art a permis de cerner les enjeux du paradigme SDN, de comprendre le protocole OpenFlow 1.3, et de justifier le choix du contrôleur Ryu pour ce projet.

L'environnement de simulation est désormais pleinement opérationnel : Mininet, Ryu, Open vSwitch et les outils de génération de trafic (iperf3, hping3, Scapy, Wireshark) sont installés et validés. La topologie de référence (arbre, profondeur 2, fanout 4, 16 hôtes) a été déployée et testée avec succès, obtenant un taux de livraison de 100% (240/240 paquets).

Les prochaines étapes porteront sur le développement des modules applicatifs : d'abord le module de gestion dynamique du trafic (Phase 3), puis le module de détection et mitigation des attaques réseau (Phase 4).

Bibliographie

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, *OpenFlow : Enabling Innovation in Campus Networks*, ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69–74, 2008.
- [2] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, *Software-Defined Networking : A Comprehensive Survey*, Proceedings of the IEEE, vol. 103, no. 1, pp. 14–76, 2015.
- [3] S. Sezer, S. Scott-Hayward, P. K. Chouhan, B. Fraser, D. Lake, J. Finnegan, N. Viljoen, M. Miller, and N. Rao, *Are We Ready for SDN? Implementation Challenges for Software-Defined Networks*, IEEE Communications Magazine, vol. 51, no. 7, pp. 36–43, 2013.
- [4] R. Braga, E. Mota, and A. Passito, *Lightweight DDoS Flooding Attack Detection Using NOX/OpenFlow*, IEEE LCN, 2010.
- [5] S. Shin, Y. Song, T. Lee, S. Lee, J. Chung, P. Porras, V. Yegneswaran, J. Noh, and B. B. Kang, *AVANT-GUARD : Scalable and Vigilant Switch Flow Management in Software-Defined Networks*, ACM CCS, 2013.